AWS Cloud Computing Practical

Experiment: Configuring an Amazon S3 Gateway VPC Endpoint

Date: 28th October 2026

1 Aim

To design and configure an Amazon Virtual Private Cloud (VPC) in AWS with public and private subnets, establish internet connectivity using an Internet Gateway and NAT Gateway, launch EC2 instances in both subnets, and configure an S3 Gateway VPC Endpoint to enable secure and private access to Amazon S3 from the private EC2 instance.

2 Topology Diagram

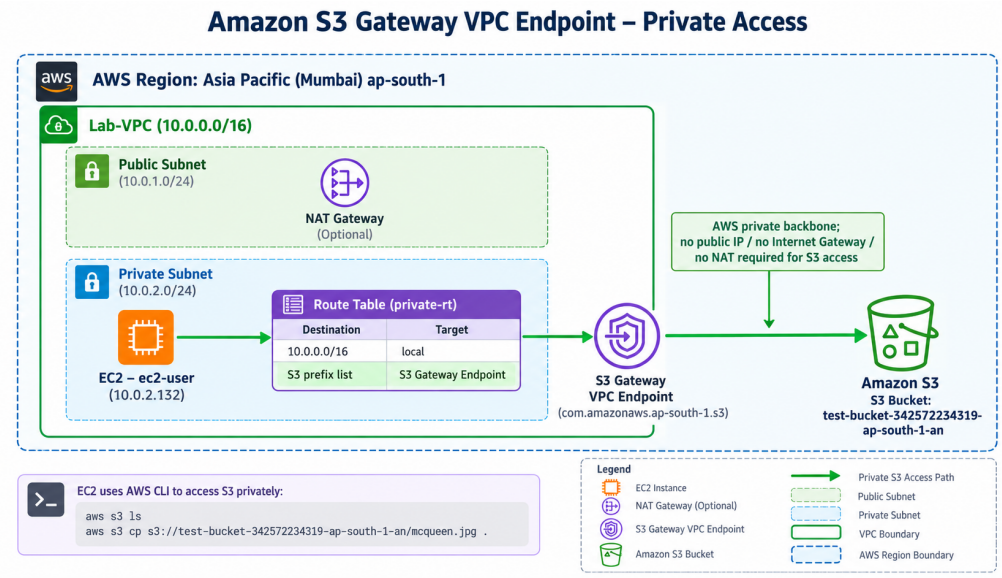


Figure 1: AWS VPC Architecture with Public and Private Subnets

3 Theory

Amazon Virtual Private Cloud (Amazon VPC) is a logically isolated virtual network within the AWS Cloud that allows users to configure IP address ranges, subnets, route tables, and network connectivity according to their requirements.

A subnet is a subdivision of a VPC associated with a specific Availability Zone. A public subnet contains a route to an Internet Gateway, enabling internet communication for resources configured with public IP addresses. A private subnet does not have a direct route to an Internet Gateway and is typically used to host backend resources.

An Internet Gateway (IGW) enables communication between a VPC and the internet. A NAT Gateway allows instances in a private subnet to initiate outbound connections to the internet while preventing unsolicited inbound connections from external networks.

Amazon EC2 (Elastic Compute Cloud) provides scalable virtual computing resources within the VPC. Security groups control inbound and outbound traffic to instances, while SSH enables secure remote administration.

AWS Command Line Interface (AWS CLI) allows users to interact with AWS services through commands executed in a terminal. It can be configured using AWS credentials and used to manage and access resources such as Amazon S3.

A VPC Gateway Endpoint provides private connectivity between resources within a VPC and supported AWS services without requiring traffic to traverse the public internet. An S3 Gateway Endpoint enables EC2 instances in a private subnet to access Amazon S3 through the AWS network using the associated route table, eliminating the need for a NAT Gateway for S3 traffic.

4 Procedure

1. **Create a VPC**

   Navigate to the AWS Management Console and open the VPC service. Select **Your VPCs** and click on **Create VPC**.

   Select the **VPC only** option and configure the following parameters:

   - Name: Lab-VPC
   - IPv4 CIDR block: 10.0.0.0/16
   - IPv6 CIDR block: No IPv6 CIDR block
   - Tenancy: Default

   Click on **Create VPC** to create the VPC.

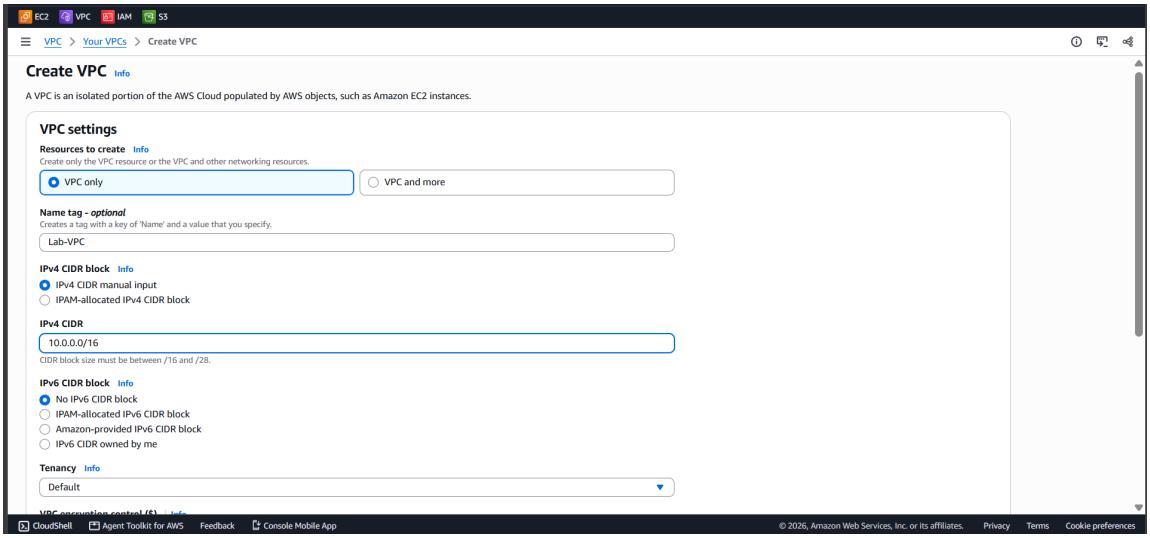


Figure 2: Configuration of VPC settings

2. Verify VPC Creation

After successful creation, navigate to the **Your VPCs** section and verify that the newly created Lab-VPC is available.

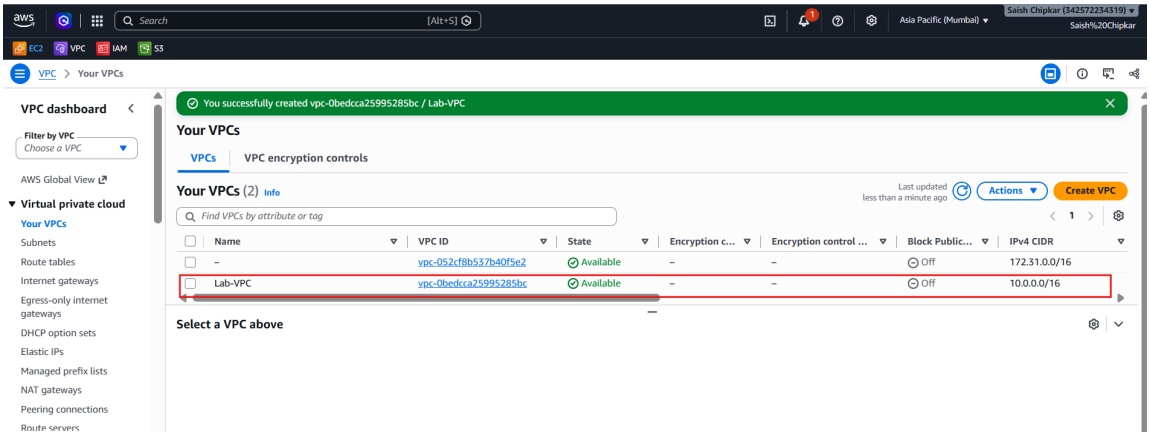


Figure 3: Successful creation of Lab-VPC

3. Create the Public Subnet

Navigate to the **Subnets** section and click on **Create subnet**. Select the Lab-VPC and configure the following parameters:

- Subnet name: Public-subnet
- Availability Zone: ap-south-1a (Mumbai)
- IPv4 VPC CIDR block: 10.0.0.0/16
- IPv4 subnet CIDR block: 10.0.1.0/24

Click on **Create subnet**.

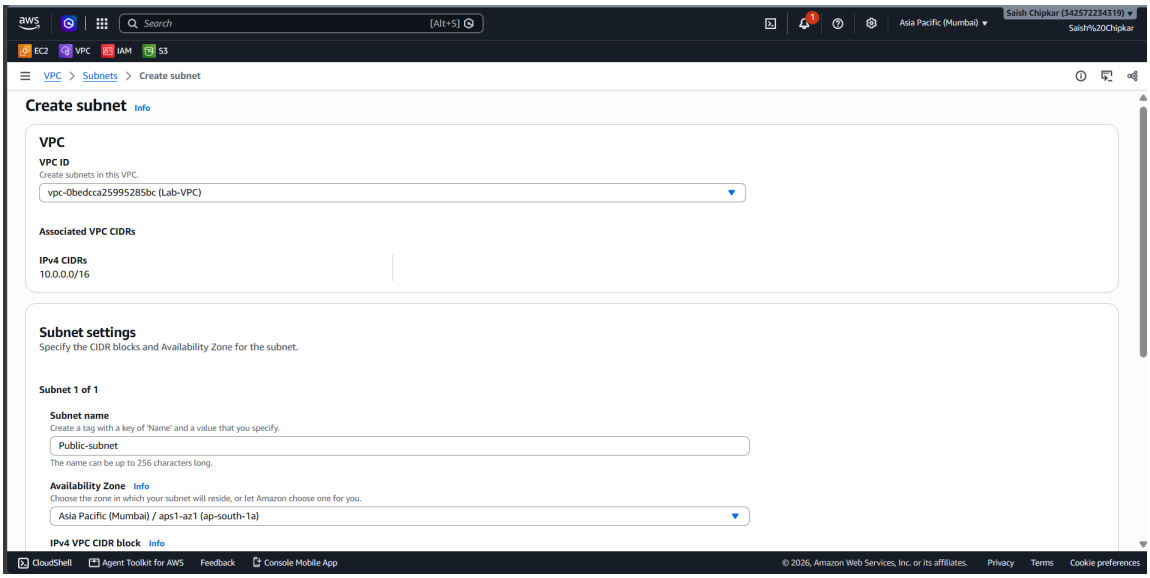


Figure 4: Configuration of the public subnet

4. **Verify Public Subnet Creation**

Verify that the public subnet has been successfully created within Lab-VPC.

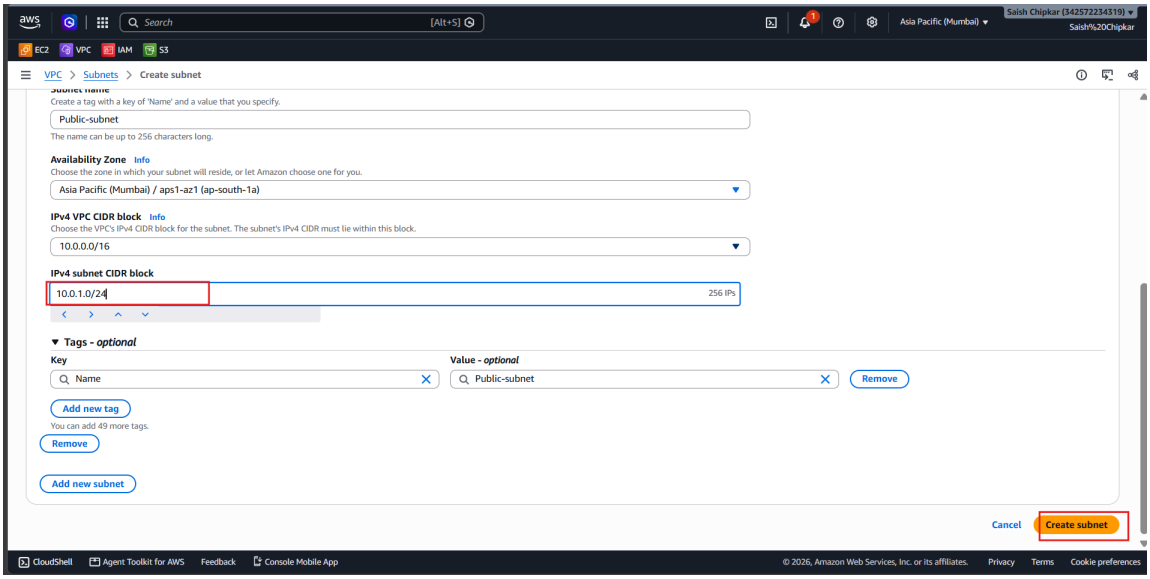


Figure 5: Creation of the public subnet

5. **Create the Private Subnet**

Click on **Create subnet** again and select the same Lab-VPC. Configure the following parameters:

- Subnet name: Private-subnet

- Availability Zone: ap-south-1b (Mumbai)
- IPv4 VPC CIDR block: 10.0.0.0/16
- IPv4 subnet CIDR block: 10.0.2.0/24

Click on **Create subnet**.

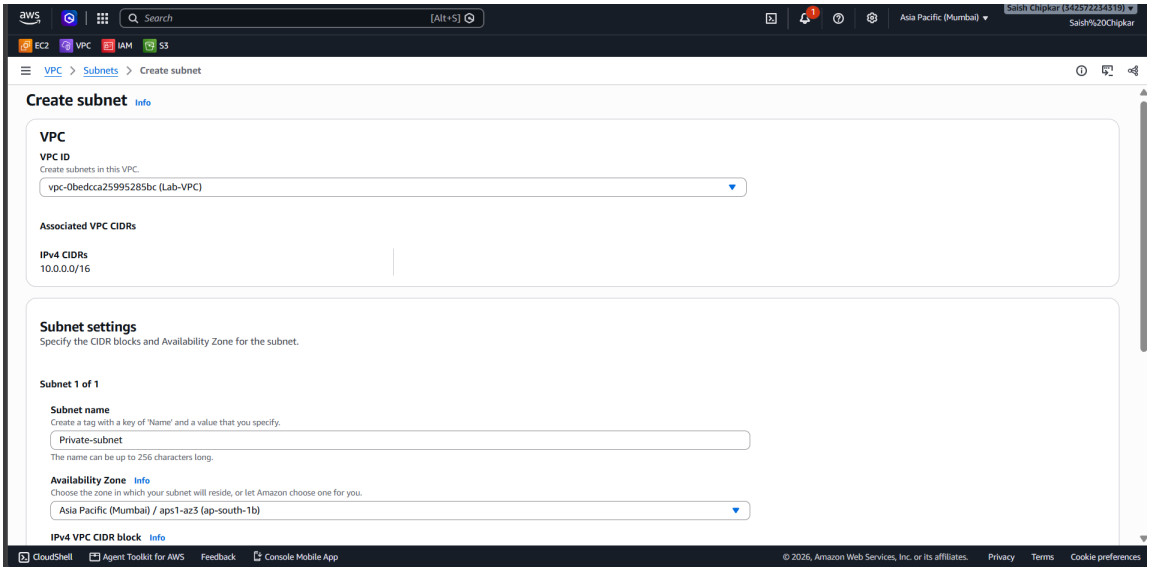


Figure 6: Configuration of the private subnet

6. Verify Private Subnet Creation

Verify that the private subnet has been successfully created within Lab-VPC.

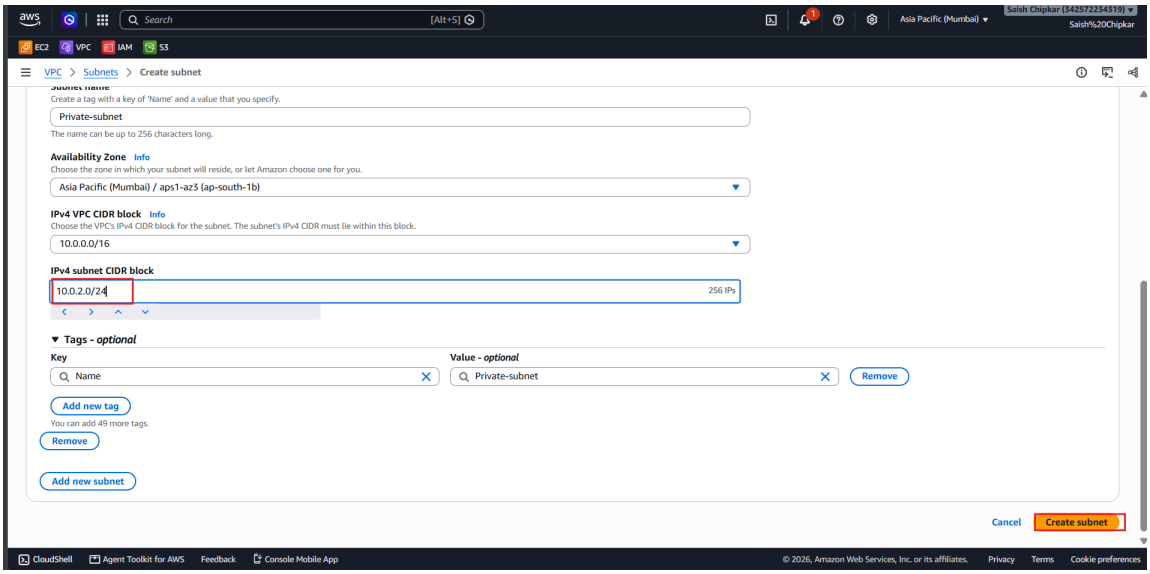


Figure 7: Creation of the private subnet

7. Create an Internet Gateway

Navigate to the **Internet Gateways** section in the VPC dashboard and click on **Create internet gateway**.

Specify the following configuration:

- Name: lab-igw

Click on **Create internet gateway** to create the Internet Gateway.

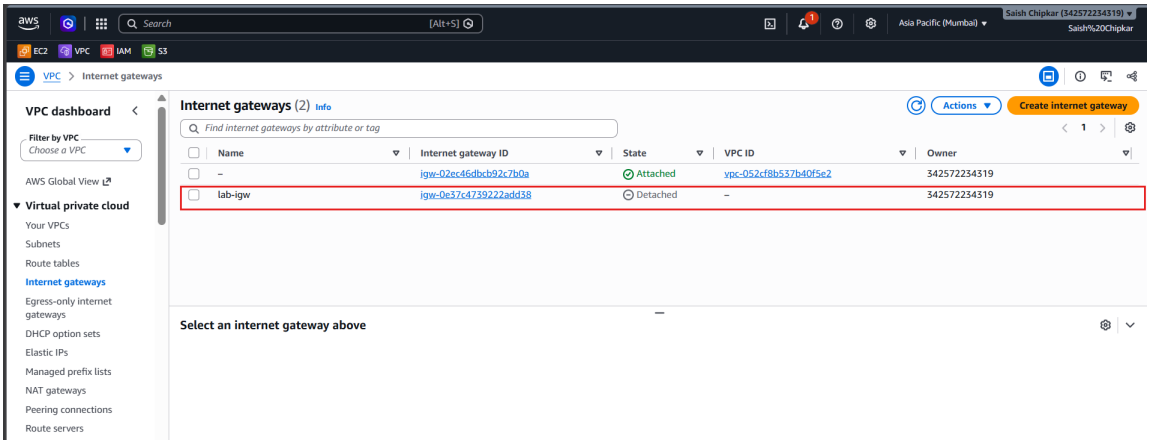
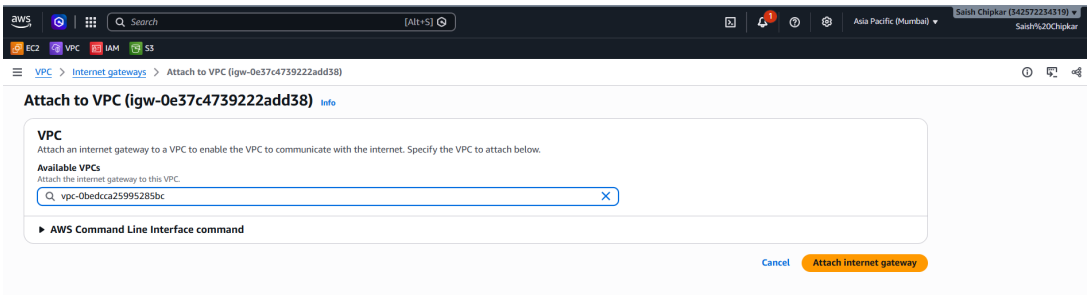


Figure 8: Creation of the Internet Gateway

Note: The Internet Gateway is initially created in a detached state. It must be attached to Lab-VPC and configured in the route table in subsequent steps to enable internet connectivity for the public subnet.

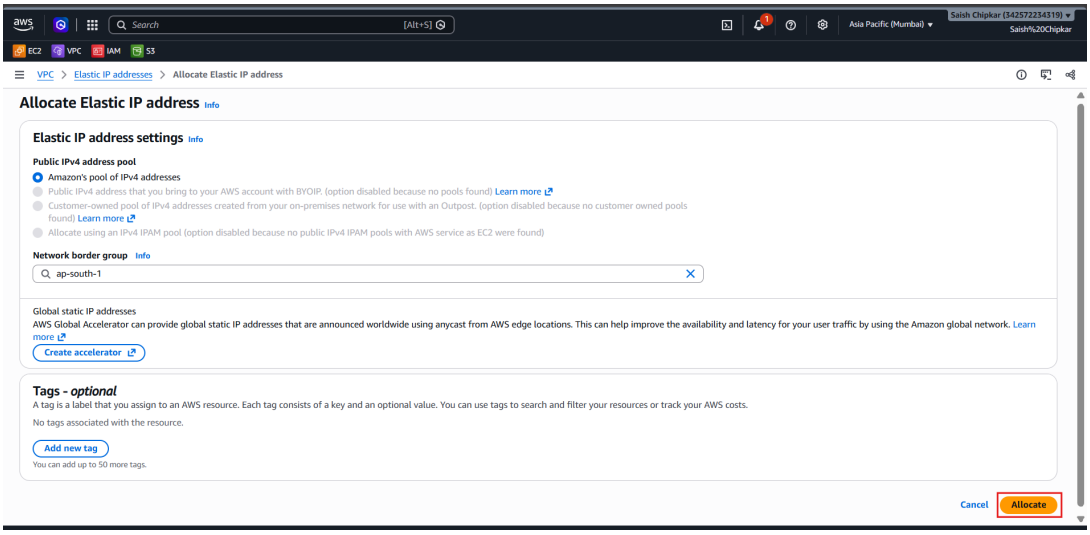
8. **Attach the Internet Gateway to the VPC:**

   Select the created Internet Gateway and click **Attach to VPC**. Choose the Lab-VPC and confirm the attachment.



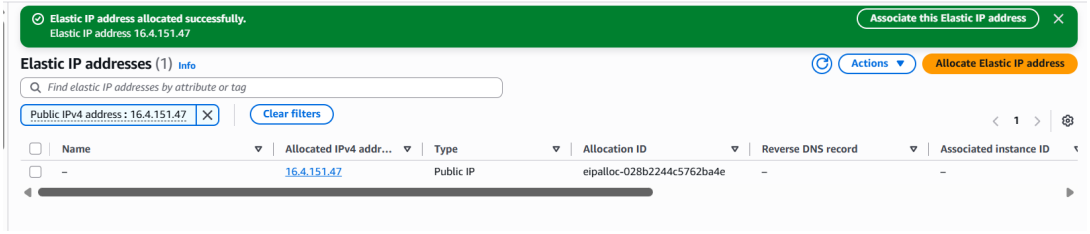
9. **Allocate an Elastic IP Address:**

   Navigate to Elastic IP addresses under the VPC dashboard. Click **Allocate Elastic IP address**, retain the default Amazon IPv4 address pool and click **Allocate**.



10. **Verify Elastic IP Allocation:**

    After successful allocation, verify the allocated public IPv4 address in the Elastic IP addresses dashboard.

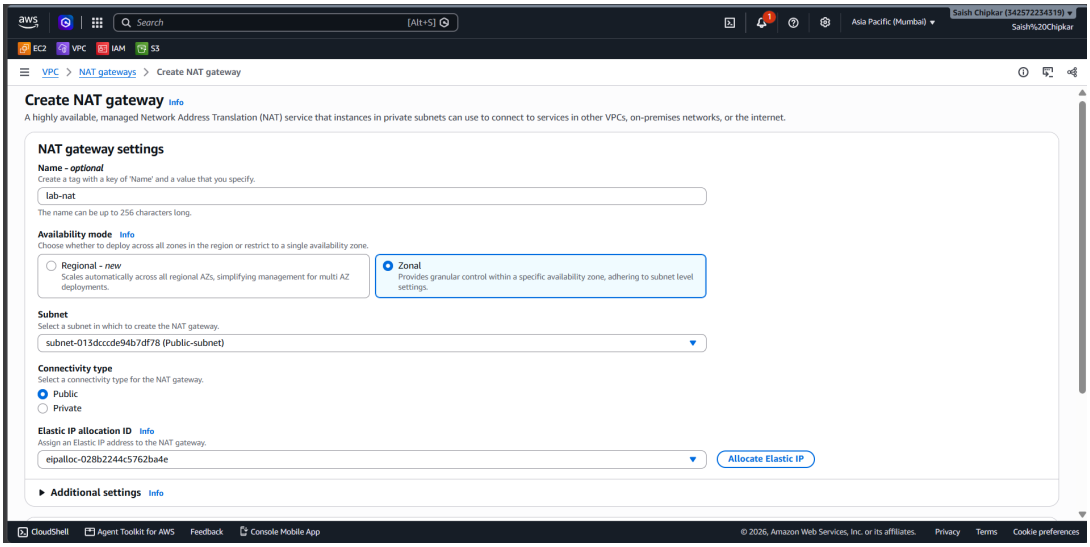


11. **Create a NAT Gateway:**

    Navigate to NAT Gateways and click **Create NAT Gateway**. Configure the following settings:

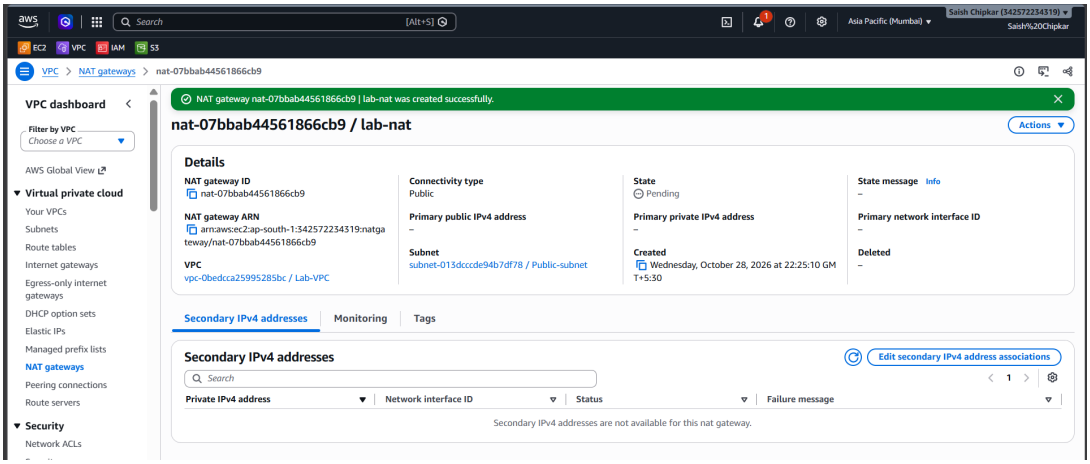
    - Name: lab-nat
    - Availability mode: Zonal
    - Subnet: Public-subnet
    - Connectivity type: Public
    - Elastic IP allocation ID: Select the allocated Elastic IP.

    Click **Create NAT Gateway** to proceed.



12. **Verify NAT Gateway Creation:**

    After creation, verify the NAT Gateway details, including its VPC, subnet, connectivity type, and current state. Wait until its state changes to Available.

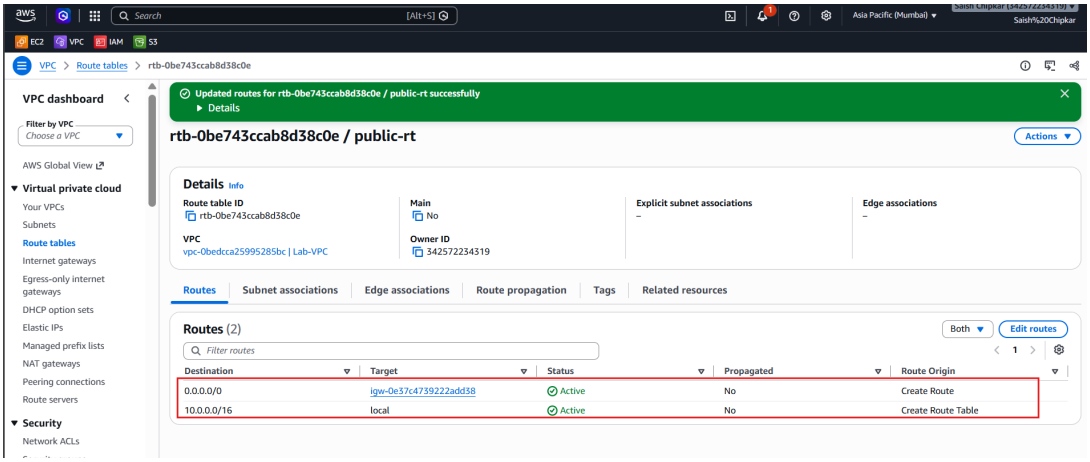


13. **Configure the Public Route Table:**

    Navigate to Route Tables and select the public route table associated with Lab-VPC. Under the Routes tab, click **Edit routes** and add the following route:

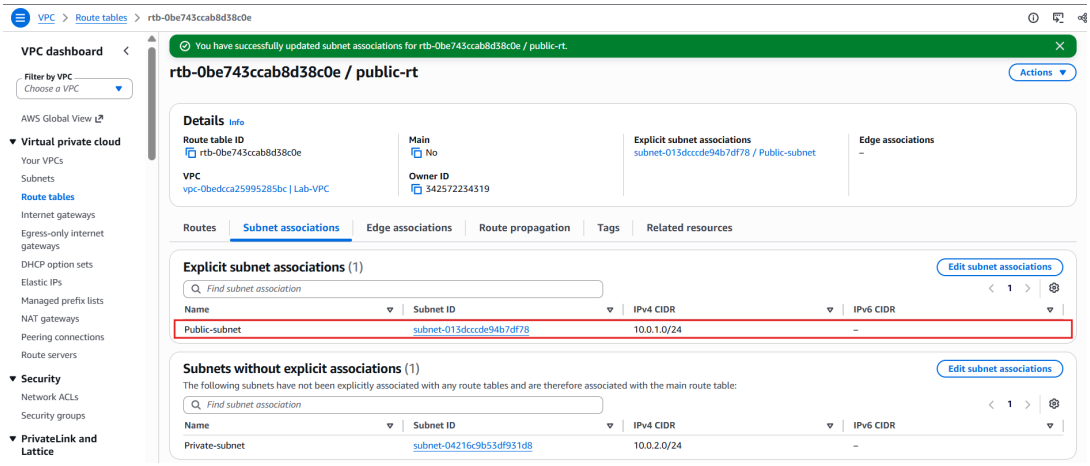
    - Destination: `0.0.0.0/0`
    - Target: Internet Gateway attached to Lab-VPC

    Save the changes.



14. **Associate the Public Subnet with the Route Table:**

    Navigate to the Subnet associations tab of the public route table. Click **Edit subnet associations**, select Public-subnet, and save the association.



Step 15: Configure Routes in the Private Route Table

Navigate to the private route table and click on **Edit routes**. Add the following route to configure outbound internet connectivity through the NAT Gateway.

- Destination: `0.0.0.0/0`
- Target: NAT Gateway

Retain the local route for communication within the VPC. Save the changes.

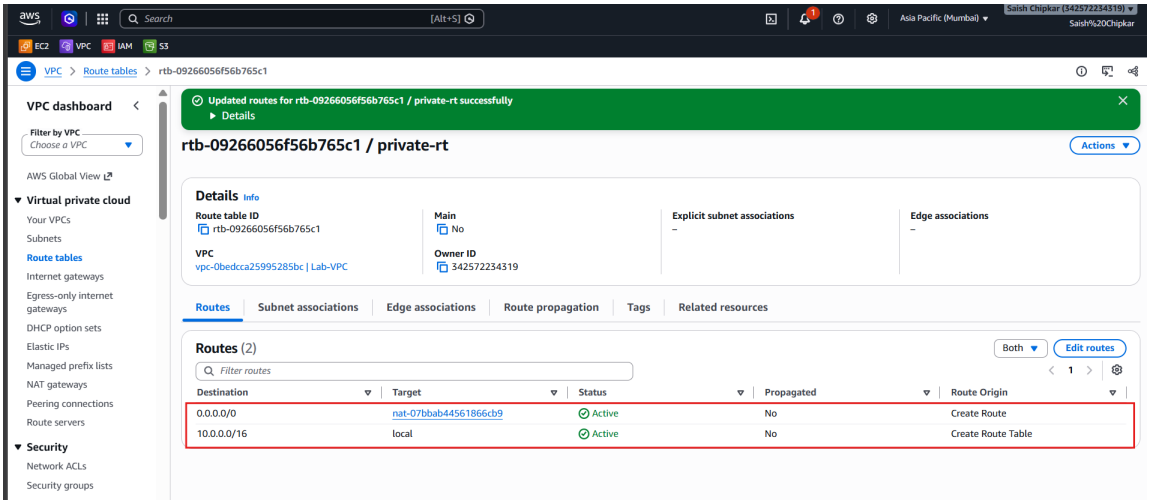


Figure 9: Configuring the private route table with a NAT Gateway

Step 16: Associate the Private Subnet with the Route Table

Navigate to the **Subnet associations** tab of the private route table. Click on **Edit subnet associations** and select the private subnet created earlier.

Save the association to ensure that the private subnet uses the configured routing table.

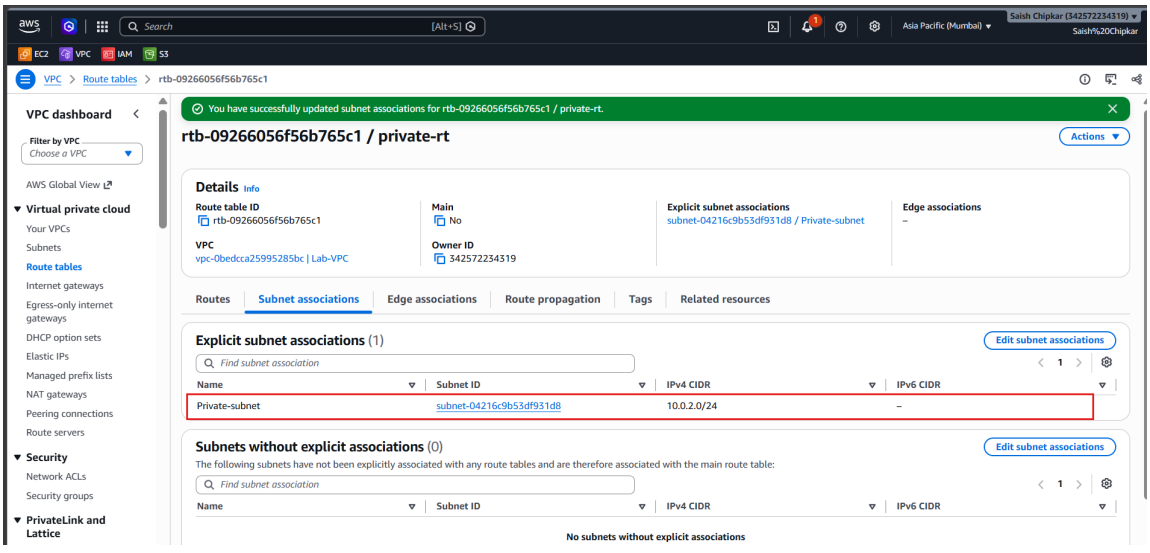


Figure 10: Associating the private subnet with the private route table

Step 17: Launch the Public EC2 Instance

Navigate to the Amazon EC2 console and click on **Launch instance**.

Configure the instance with the following details:

- Name: `public-server`
- AMI: Amazon Linux
- Instance type: `t3.micro`
- Key pair: Select the previously created key pair

Proceed to configure the network settings.

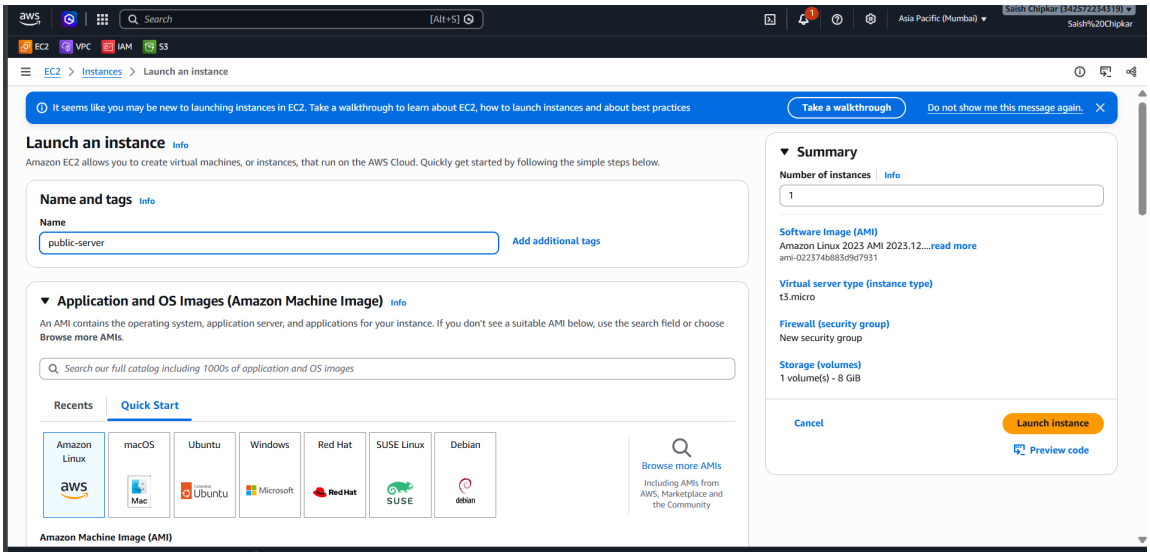


Figure 11: Configuring the public EC2 instance

Step 18: Configure Network Settings for the Public Instance

Under **Network settings**, configure the following:

- VPC: Select the created Lab-VPC
- Subnet: Select the public subnet
- Auto-assign public IP: Enable
- Security group: Create a new security group
- Inbound rule: Allow SSH (TCP port 22)

After verifying the configuration, click on **Launch instance**.

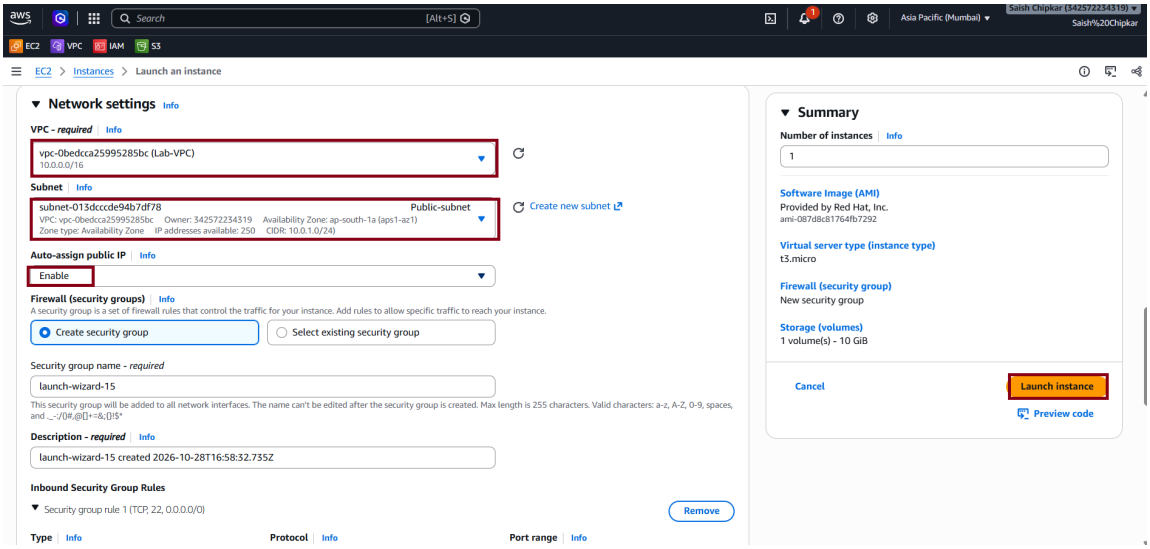


Figure 12: Network configuration for the public EC2 instance

Step 19: Launch the Private EC2 Instance

Similarly, navigate to the EC2 console and select **Launch instance** to create the private server.

Configure the following details:

- Name: `private-server`
- AMI: Red Hat Enterprise Linux
- Instance type: `t3.micro`
- Key pair: Select the previously created key pair

Proceed to configure the network settings.

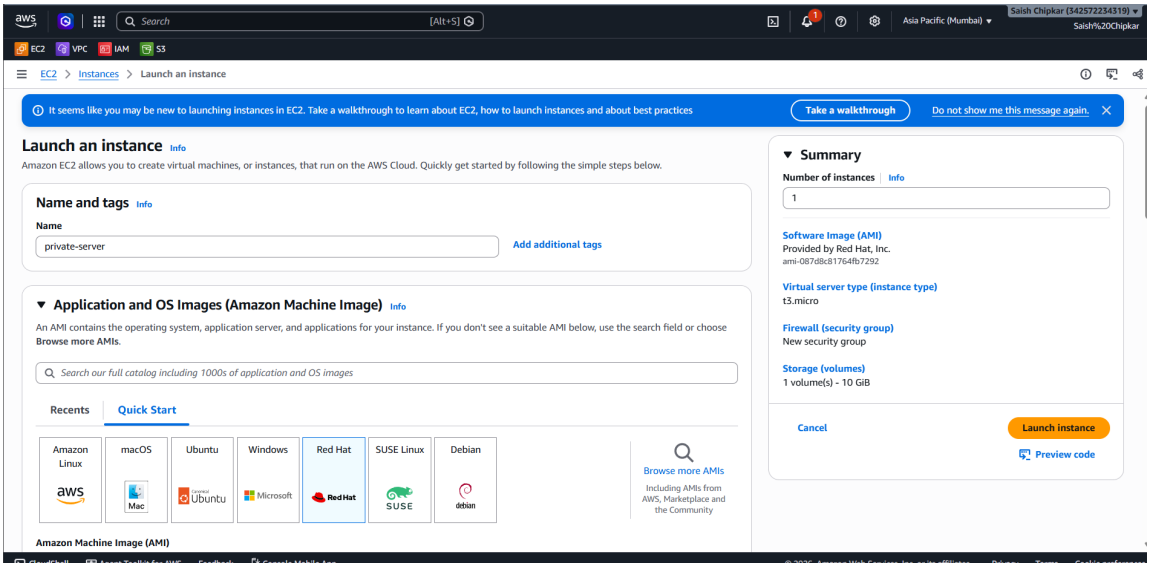


Figure 13: Configuring the private EC2 instance

Step 20: Configure Network Settings for the Private Instance

Under **Network settings**, configure the following:

- VPC: Select the created Lab-VPC
- Subnet: Select the private subnet
- Auto-assign public IP: Disable
- Security group: Create a new security group
- Inbound rule: Allow SSH (TCP port 22) as required

Verify the configuration and click on **Launch instance**.

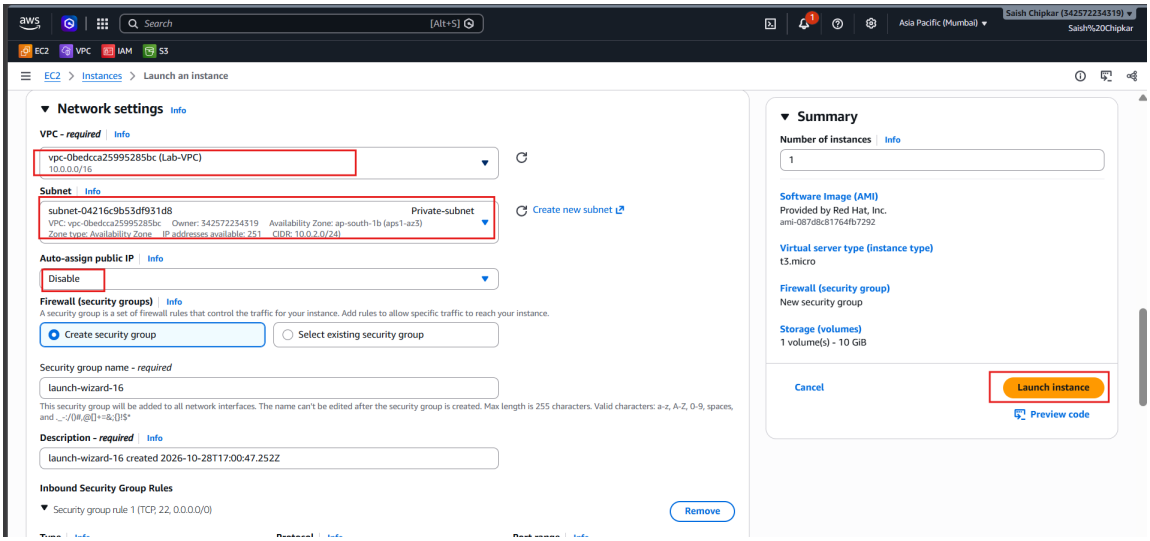


Figure 14: Network configuration for the private EC2 instance

Step 21: Transfer the SSH Key to the Public Instance

Open Windows PowerShell on the local computer and execute the following SCP command to securely transfer the private key file to the public EC2 instance.

```
scp -i "C:\Users\Saish Chipkar\Downloads\key18.pem" ‘
"C:\Users\Saish Chipkar\Downloads\key18.pem" ‘
ec2-user@13.126.80.147:/home/ec2-user
```

The key file is transferred to the public instance, allowing it to be used for SSH authentication when connecting to the private EC2 instance.

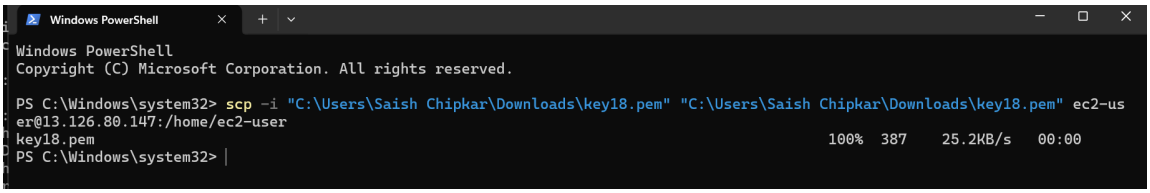


Figure 15: Transferring the SSH private key using SCP

4.1 Step 22: Connect to the Private EC2 Instance

From the public EC2 instance, connect to the private EC2 instance using SSH. The private key file must have appropriate permissions before establishing the connection.

```
sudo su
ls
chown root:root key18.pem
chmod 700 key18.pem
ssh -i "key18.pem" ec2-user@10.0.2.132
```

Accept the host authenticity prompt by entering `yes` when connecting for the first time.



Figure 16: SSH connection from the public instance to the private instance

4.2 Step 23: Install AWS CLI on the Private Instance

After successfully connecting to the private EC2 instance, install AWS CLI Version 2 using the following command.

```
curl -fsSL https://awscli.amazonaws.com/v2/install.sh | bash
```

Verify the installation by checking the AWS CLI version.

```
aws --version
```

```
[ec2-user@ip-10-0-2-132 ~]$ curl -fsSL https://awscli.amazonaws.com/v2/install.sh | bash
Resolving user-local install paths:
  HOME=/home/ec2-user
  XDG_DATA_HOME=<unset, using default>
  XDG_BIN_HOME=<unset, using default>
Installing AWS CLI 2.39.0
Downloading AWS CLI installer from https://awscli.amazonaws.com/awscli-exe-linux-x86_64-2.39.0.zip?src=script-exe
######################################################################## 100.0%
Downloading AWS CLI signature from https://awscli.amazonaws.com/awscli-exe-linux-x86_64-2.39.0.zip.sig?src=script-exe
######################################################################## 100.0%
GPG signature verified.
You can now run: /home/ec2-user/.local/bin/aws --version
Tip: run 'aws configure agent-toolkit' to set up AWS skills and the AWS MCP server for your AI coding agent(s).
AWS CLI 2.39.0 installed to /home/ec2-user/.local/share/aws-cli
Ensure /home/ec2-user/.local/bin is on your PATH. When multiple installs exist, PATH order determines which 'aws' run
s.
[ec2-user@ip-10-0-2-132 ~]$ aws --version
aws-cli/2.39.0 Python/3.14.8 Linux/6.12.0-211.53.1.el10_2.x86_64 script-exe/x86_64.rhel.10
[ec2-user@ip-10-0-2-132 ~]$
```

Figure 17: Installation and verification of AWS CLI on the private instance

4.3 Step 24: Navigate to AWS Security Credentials

Log in to the AWS Management Console using the root user. Click on the account name in the top-right corner and select **Security credentials** from the dropdown menu.

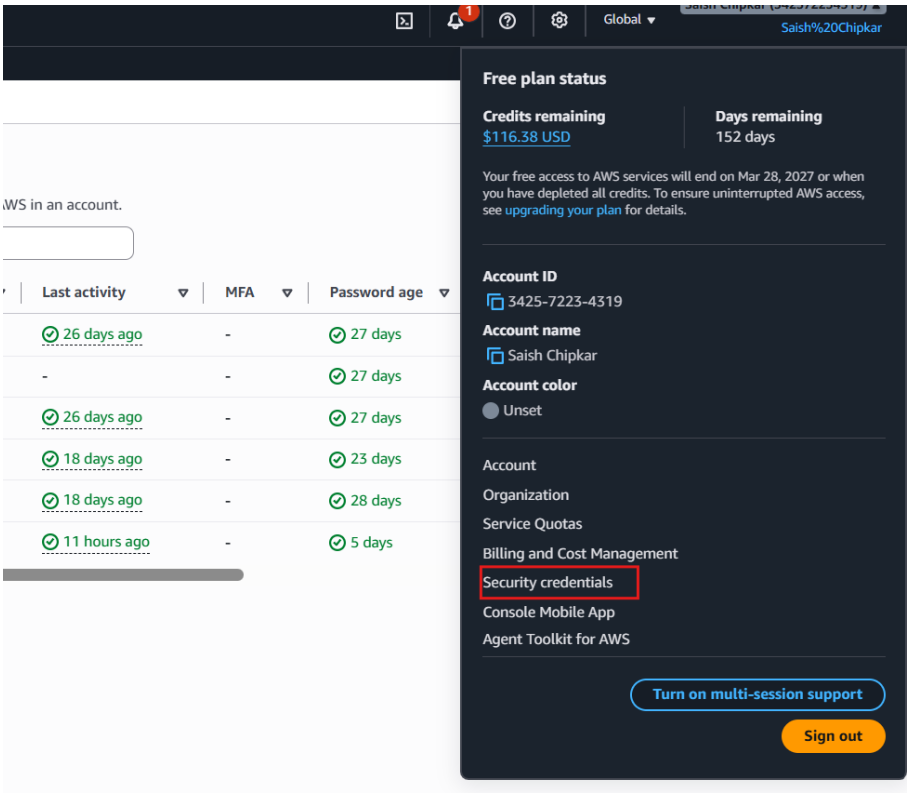


Figure 18: Navigating to Security Credentials in the AWS Console

4.4 Step 25: Create an Access Key

In the Security Credentials page, navigate to the **Access keys** section and click on **Create access key**.

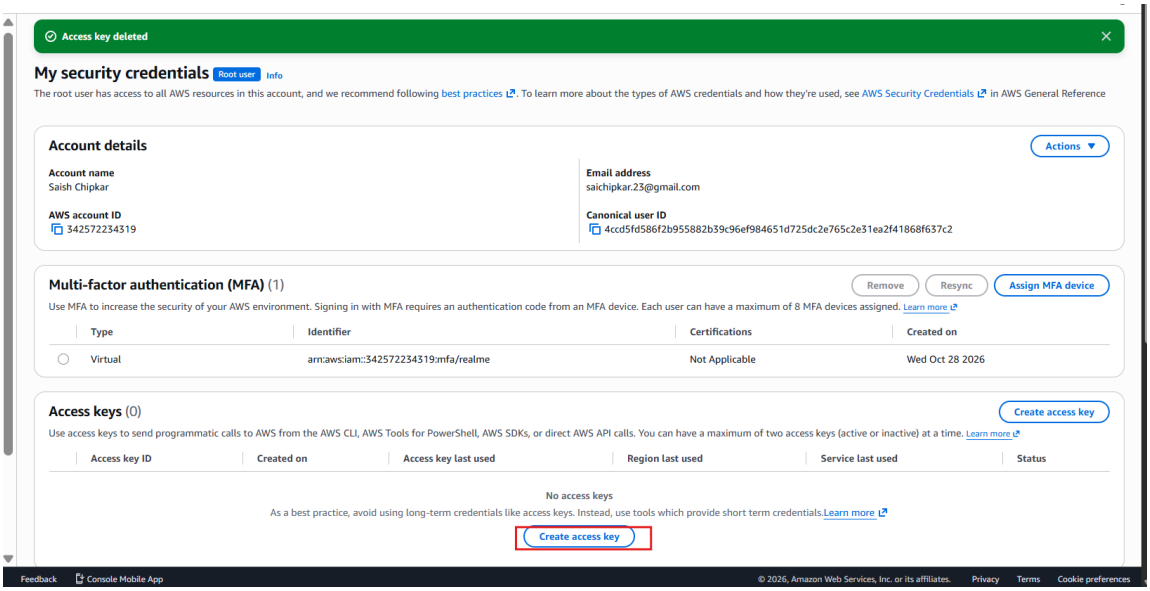


Figure 19: Access keys section under AWS Security Credentials

4.5 Step 26: Acknowledge the Access Key Warning

AWS displays a warning stating that creating access keys for the root user is not recommended. Select the acknowledgement checkbox to confirm that the risks are understood and click on **Create access key**.

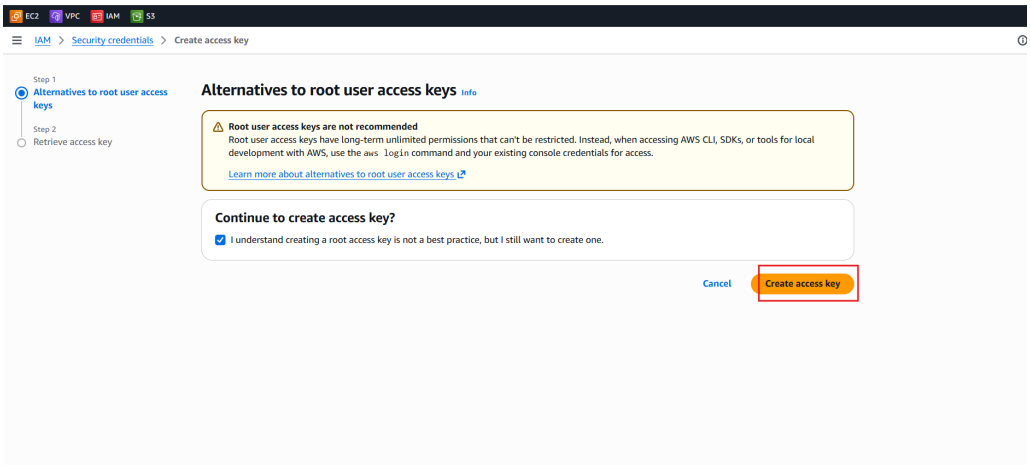


Figure 20: Acknowledging the risks of creating root user access keys

4.6 Step 27: Retrieve the Access Key and Secret Access Key

Once the access key is created, AWS displays the Access Key ID and Secret Access Key. These credentials are required to configure AWS CLI.

Download the credentials securely or copy them to a secure location. The Secret Access Key is displayed only once during creation.

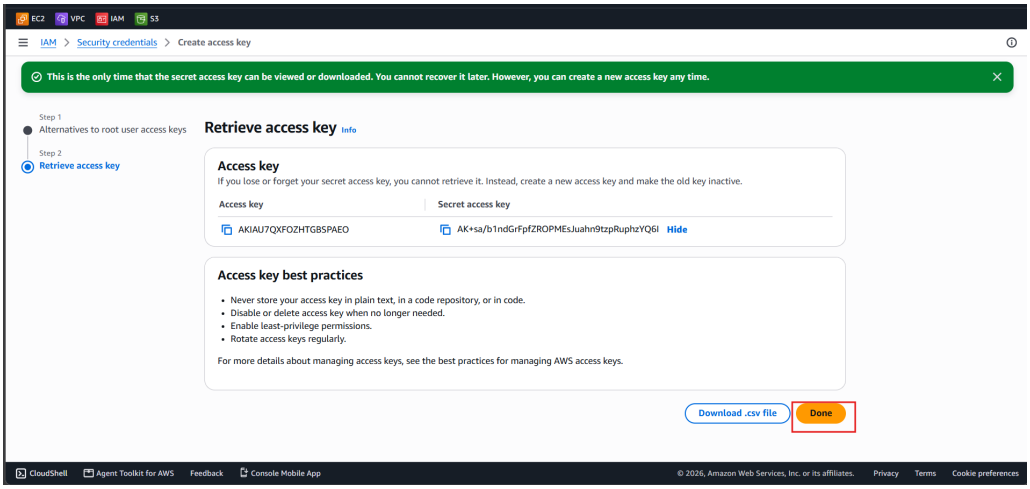


Figure 21: Retrieving the Access Key ID and Secret Access Key

4.7 Step 28: Verify the S3 Bucket

Navigate to the Amazon S3 service in the AWS Management Console. Open the previously created S3 bucket and verify that the uploaded object is present in the bucket.

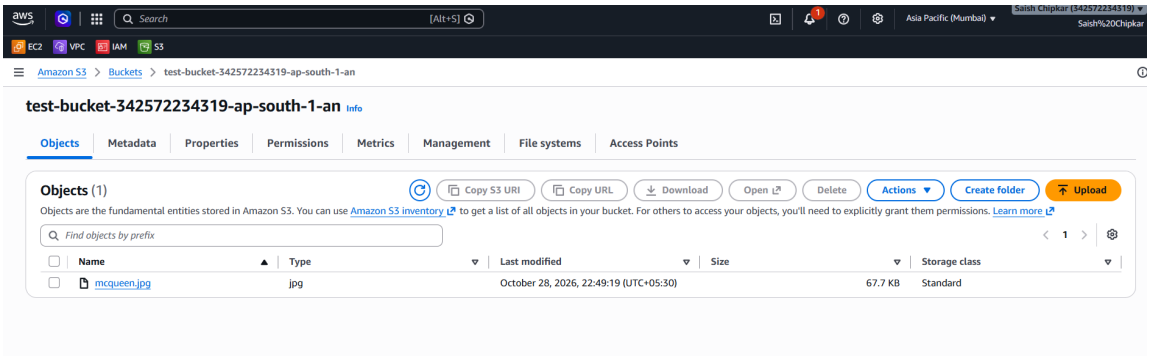


Figure 22: Verifying the uploaded image in the Amazon S3 bucket

4.8 Creating an S3 Gateway VPC Endpoint

(a) Navigate to the **VPC Dashboard** in the AWS Management Console. Under the **PrivateLink and Lattice** section, select **Endpoints** and click on **Create endpoint**.

```
[ec2-user@ip-10-0-2-132 ~]$ aws configure
AWS Access Key ID [****************PAEO]: AKIAU7QXFOZHRWF2TQTA
AWS Secret Access Key [****************YQ61]: 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
Default region name [ap-south-1]:
Default output format [json]:
[ec2-user@ip-10-0-2-132 ~]$ aws s3 ls
2026-10-28 17:18:16 test-bucket-342572234319-ap-south-1-an
[ec2-user@ip-10-0-2-132 ~]$
```

Figure 23: Navigating to VPC Endpoints

(b) Enter the endpoint name as `s3-endpoint`. Under the **Type** section, select **AWS services** to create an endpoint for an AWS service.

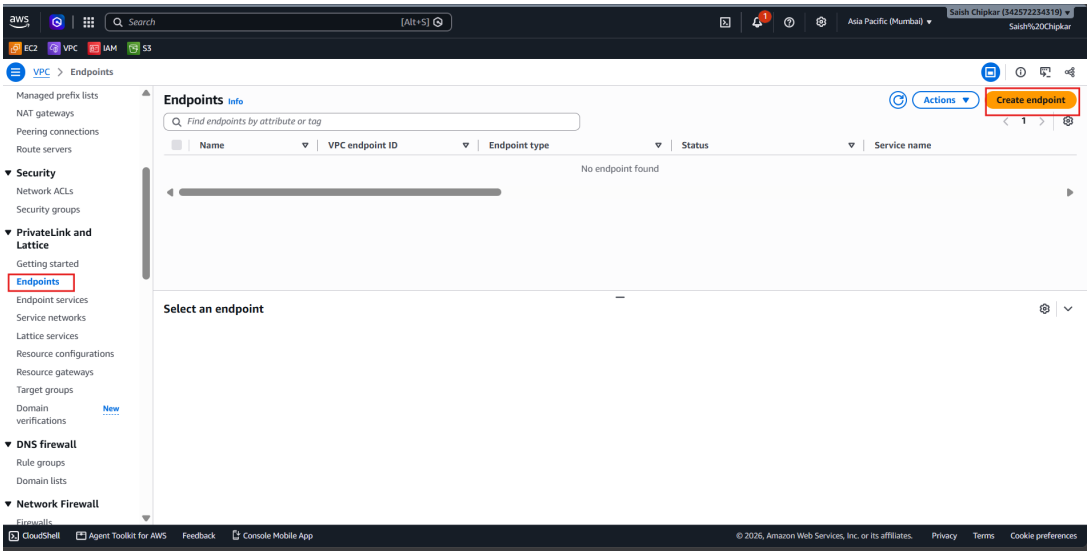


Figure 24: Specifying the endpoint name and type

(c) Enable the **Cross Region endpoint** option and select **Asia Pacific (Mumbai) (ap-south-1)** as the service region. Search for the S3 service and select the service name `com.amazonaws.ap-south-1.s3` with the **Gateway** endpoint type.

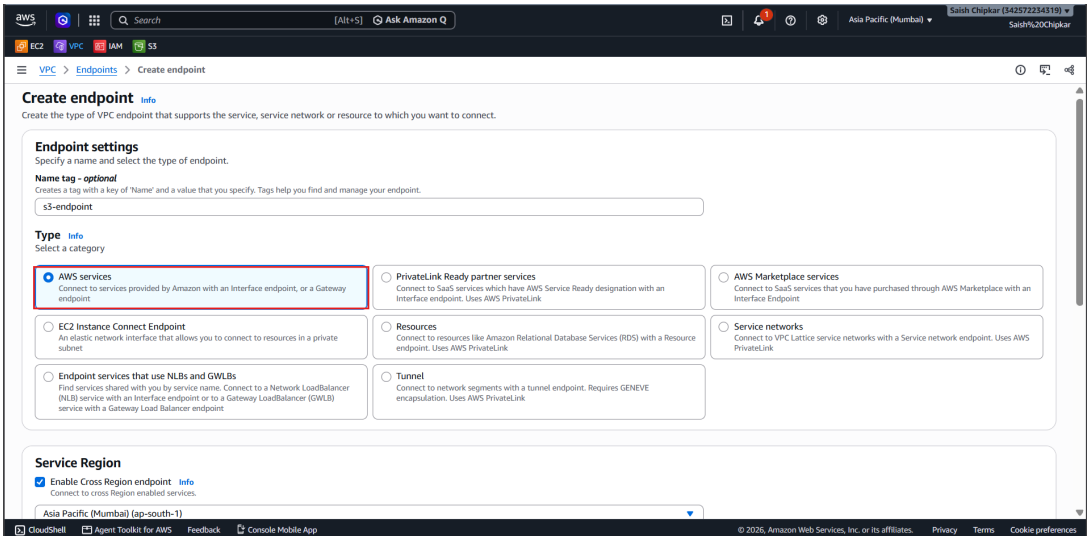


Figure 25: Selecting the Amazon S3 Gateway service

(d) Under **Network settings**, select the previously created **Lab-VPC**. In the **Route tables** section, select `private-rt`, which is associated with the private subnet. Select the **Full access** option under the endpoint policy and proceed with creating the endpoint.

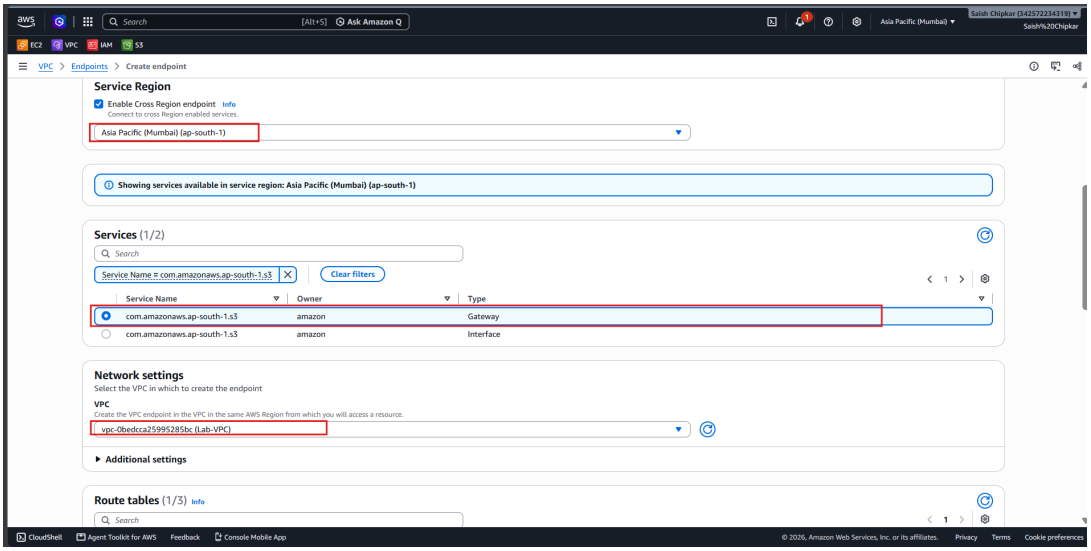


Figure 26: Configuring the VPC and private route table

(e) Navigate to the **private-rt** route table and edit its routes. Remove the default route `0.0.0.0/0` pointing to the NAT Gateway and save the changes.

The S3 Gateway Endpoint automatically adds a route for the S3 service through the endpoint, allowing the private instance to access S3 without requiring a NAT Gateway for this traffic.

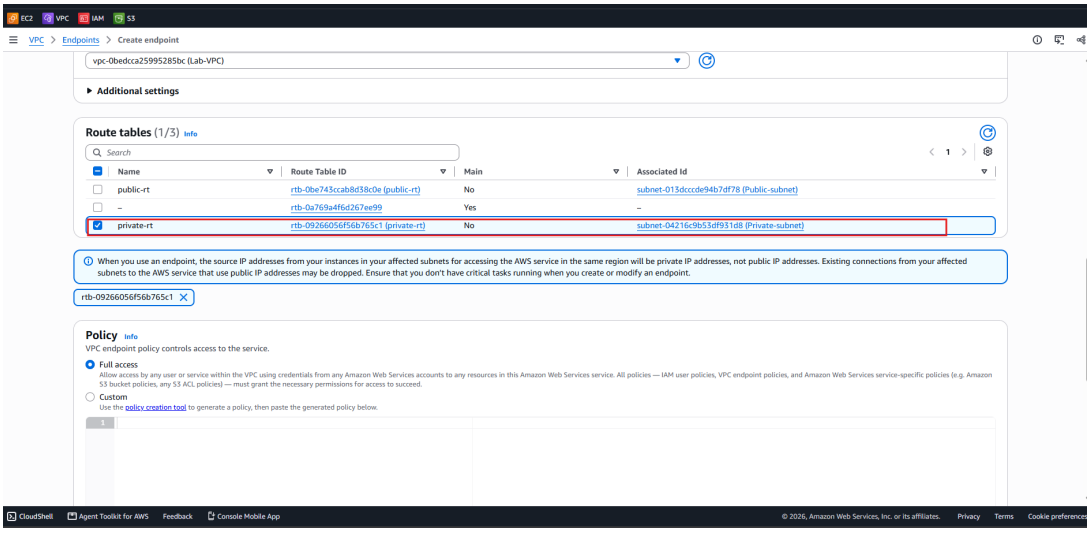


Figure 27: Updating the private route table

4.9 Accessing Amazon S3 Using AWS CLI

(a) Connect to the EC2 instance in the private subnet using SSH through the configured access mechanism. Execute the following command to list the available S3 buckets:

```
aws s3 ls
```

The command successfully lists the S3 bucket, demonstrating connectivity between the private EC2 instance and Amazon S3 through the Gateway VPC Endpoint.

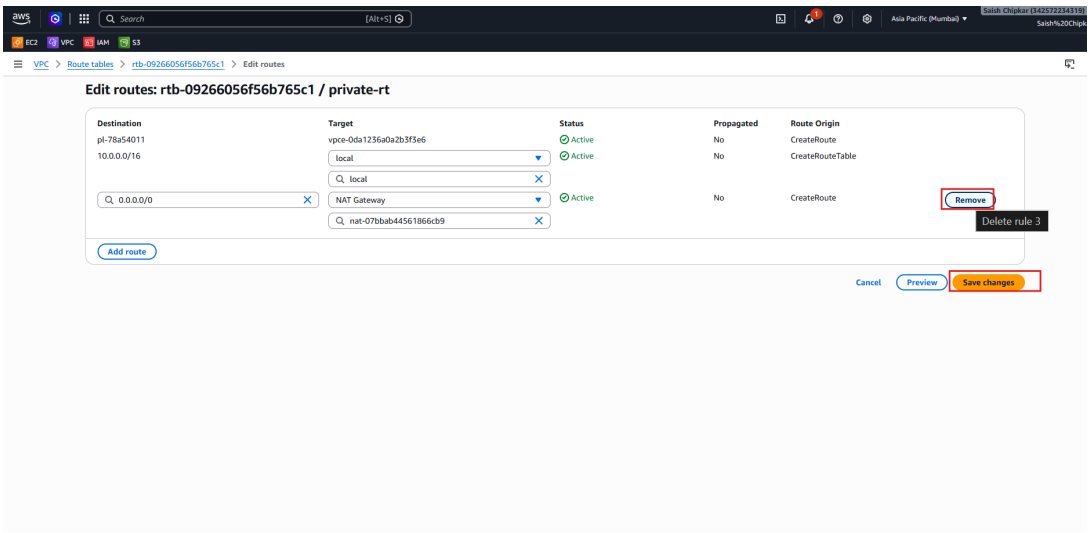


Figure 28: Listing S3 buckets using AWS CLI

(b) Execute the following command to list the objects stored inside the S3 bucket:

```
aws s3 ls s3://test-bucket-342572234319-ap-south-1-an/
```

(c) Download the required object from the S3 bucket to the local directory of the EC2 instance using the following command:

```
aws s3 cp s3://test-bucket-342572234319-ap-south-1-an/mcqueen.jpg \
/home/ec2-user/mcqueen.jpg
```

(d) Verify the downloaded file using the following command:

```
ls -l
```

The downloaded image file is successfully stored in the EC2 instance's local directory.

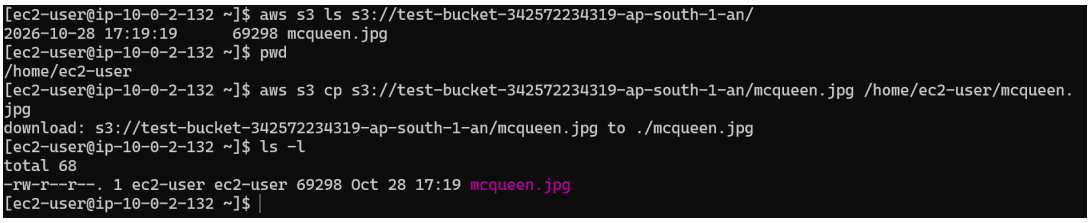


Figure 29: Downloading an S3 object to the EC2 instance

4.10 Result

Successfully created and configured an Amazon S3 Gateway VPC Endpoint for the private subnet. Verified that the EC2 instance can list S3 buckets, access their objects, and download files without requiring internet connectivity through a NAT Gateway.

5 Result

Thus, an Amazon VPC named Lab-VPC was successfully created and configured with public and private subnets, route tables, an Internet Gateway, and a NAT Gateway. Public and private EC2 instances were launched, and SSH connectivity to the private instance was established through the public instance. AWS CLI was installed and configured on the private EC2 instance. An S3 Gateway VPC Endpoint was successfully created and

associated with the private route table. The private EC2 instance successfully listed S3 buckets, accessed bucket objects, and downloaded an image from Amazon S3 without requiring NAT Gateway connectivity for S3 access.